PDF Batch Tools

1. What was built and why

A Python command-line tool applies a specified whole-page transformation to a list of PDF files while preserving the originals and producing a per-file evidence manifest. The target problem is repetitive document processing: a user has many files and needs the same explicitly defined operation applied consistently, without accidentally replacing source files or losing track of which output came from which input.

This is a controlled PDF processing demonstration, not a drawing editor, OCR system or architectural service. In particular, reflecting an entire PDF page reflects its text too. Moving diagram geometry while keeping labels readable is a different task and is not implemented.

2. Input, outputs and supported operations

The input is a version-1 JSON manifest with jobs containing input, output and operation. Each input resolves inside the manifest directory; each output is a simple PDF filename. Output names must be unique even when compared without case. The output directory must not already exist.

Operation	Coordinate mapping on a page of width W and height H
mirror_horizontal	x becomes W - x; y is unchanged
mirror_vertical	x is unchanged; y becomes H - y
rotate_180	x becomes W - x; y becomes H - y
copy	Coordinates are unchanged; the PDF is reserialized

The tool writes the converted PDFs and manifest.json containing input/output SHA-256 values, filenames, requested operations, page counts and page dimensions. "Copy" means a geometrically unchanged output, not byte-for-byte preservation of PDF serialization or all metadata.

3. Implementation method

pypdf parses each supported input, clones its pages into a new writer and adds the requested affine transformation to the page content. The result is reopened to check structural support and page count. All jobs are prepared and validated before output directory creation, so a preflight error does not create a partially converted batch.

Inputs must be unencrypted PDFs with 1-100 pages, positive page dimensions, zero-origin equal media/crop boxes, no preexisting page rotation and default user units. Forms, annotations, page/document actions and named/embedded objects require separate handling and are rejected. Limits are 20 MiB per input, 500 jobs and 100 MiB of prepared output. These are configured processing limits, not a security sandbox.

4. Test data and evaluation procedure

demo.py uses ReportLab to generate 80 one-page PDFs at 600 by 420 points. They share one synthetic schematic template containing asymmetric room geometry, labels and a colored marker, with different file numbers. Each of the four operations is applied to 20 files. These are 80 instances of one template, not 80 independently sourced document types or client drawings.

Structural tests also use a two-page generated document and deliberately constructed unsupported inputs. This separates multi-page structure checks from the one-page visual corpus; the visual comparison does not claim coverage of arbitrary multi-page layouts.

For the visual check, render every original and result at 72 DPI with pdftoppm. Create the expected image by applying the specified mirror/rotation to the rendered original using Pillow. Compare that expected image with the rendered output. The mean absolute RGB error is the sum of absolute channel differences divided by width x height x 3, on a 0-255 channel scale. It is not a percentage accuracy score.

The declared pass criterion is matching image dimensions and mean absolute RGB error no greater than 2.0 for every pair. Rasterization and antialiasing can produce small differences around text and lines; this tolerance was used in the recorded check, not calibrated against a broad external corpus. A low global mean error could still miss a small local defect, so it is not a guarantee of semantic or pixel-perfect fidelity.

5. Observed evidence

Check	Recorded result
Converted fixture files	80 of 80, each one page
Operation balance	20 horizontal mirrors, 20 vertical, 20 rotations, 20 copies
Rendered and compared pairs	All 80 pairs
Largest observed mean RGB error	0.544070 on the 0-255 scale
Pairs above the declared limit of 2.0	0
Automated test methods	13 passed

Tests cover all four transformations, page structure and extractable text, input preservation and output hashes, invalid/encrypted/rotated/annotated inputs, nonzero page origins, filename collisions, path traversal, unsupported operations and preflight/no-overwrite behavior. Representative outputs were also visually inspected. No processing-speed improvement, customer time saving or general PDF compatibility rate was measured.

6. Reproduce and inspect

The recorded environment used Python 3.12.14, pypdf 6.10.0, ReportLab 4.4.9 and Pillow 12.3.0 on macOS, plus pdftoppm for rasterization. Conversion requires pypdf; ReportLab creates the demo. Pillow and Poppler are additionally required for the visual verification script. Python 3.10+ is declared; other Python versions were not validated in the recorded run.

```
python -m pip install -r requirements.txt
python -m unittest -v
python demo.py --output demo-output
python -m pip install Pillow==12.3.0
python verify_rendering.py demo-output --output render-check
```

Install Poppler separately so pdftoppm is on PATH. Use new output directories. The verification script checks manifest hashes, renders the supplied one-page corpus and writes per-file measurements; it is an evaluator for this fixture format, not an untrusted-upload service.

Evidence	Location
Preflight, transformations and manifests	pdf_batch.py
Self-created document generator	demo.py
Structural and negative tests	test_pdf_batch.py
Repeatable visual comparison	verify_rendering.py
Recorded per-file visual measurements	render_verification.json
Representative original and horizontal mirror	sample_input.pdf; sample_mirrored.pdf
Recorded test run and source hashes	verification.json

7. Limits and client adaptation

Whole-page mirroring intentionally reverses text. Keeping labels readable, changing room layouts, extracting tables, OCR, interactive forms and annotation editing are outside this implementation. Rejected documents are not automatically repaired. A resource limit does not make a parser safe against every hostile PDF; the tool is not a malware scanner or operating-system sandbox.

Source files are left unchanged and existing result directories are rejected. A disk error during final writes can still leave an incomplete new directory; there is no resume transaction or upload workflow. The supplied fixture validates only a narrow family of PDFs. Before accepting client documents, confirm the precise geometric operation, inspect their PDF features, obtain permission to process them, agree on visual acceptance criteria and run a representative sample before the full batch.

8. Contribution and evidence boundary

This independent engineering project covers implementation, tests and documentation. All document content is self-created; no client files, construction drawings, architectural qualifications or production deployments are claimed. Tests establish the documented behavior on the stated examples, not universal document compatibility.